

Práctica 2

JDBC CRUD (PostgreSQL)

1. Objetivos de la práctica

- Ampliar el proyecto de la sesión 1 para soportar CRUD completo sobre la tabla libro.
- Hacer SELECT por id (findById).
- Hacer INSERT de un libro nuevo usando PreparedStatement y recuperar el id generado (getGeneratedKeys).
- Hacer UPDATE en el libro nuevo que se inserto para ponerlo como 'No disponible' y posteriormente DELETE a ese mismo libro comprobando en ambos casos las filas afectadas (rowsAffected).
- Dejar un main de prueba que ejecute un flujo completo: crear → leer → actualizar → borrar.

Al terminar la práctica, debes ser capaz de ejecutar un main que inserte un libro, imprima el id generado y demuestre que update y delete funcionan.

2. Checklist antes de empezar

Asegúrate de que la sesión 1 está funcionando:

- Tu db.properties apunta a la base de datos correcta y te conecta sin error.
- En pgAdmin, ejecuta un SELECT para comprobar que hay datos

3. PostgreSQL: reiniciar datos para practicar

Para que todos trabajemos con los mismos datos, ejecuta este script en pgAdmin:

```
TRUNCATE TABLE libro RESTART IDENTITY;

INSERT INTO libro (titulo, isbn, anio, disponible) VALUES
('Harry Potter y la Piedra Filosofal', '9780132350884', 1999, true),
('Los Pilares de la Tierra', '9780134685991', 1998, true),
('El Hobbit', '9780201633610', 1964, false);

SELECT * FROM libro ORDER BY id;
```

OJO: isbn es UNIQUE. Si intentas insertar un libro con un isbn repetido, PostgreSQL dará error (duplicate key).

4. Evolución del modelo Libro (para INSERT y UPDATE)

En la sesión 1, id era “siempre conocido” porque venía de la base de datos. Para INSERT, necesitamos crear un Libro sin id y asignarlo después.

4.1 Actualiza tu clase Libro

Ruta: src/main/java/com/dam/ad/model/Libro.java

```
package com.dam.ad.model;

public class Libro {
    private Integer id;          // ahora puede ser null antes del INSERT
    private String titulo;
    private String isbn;
    private Integer anio;
    private boolean disponible;

    // Constructor para cuando leemos de BD (id conocido)
    public Libro(int id, String titulo, String isbn, Integer anio,
boolean disponible) {
        this.id = id;
        this.titulo = titulo;
        this.isbn = isbn;
        this.anio = anio;
        this.disponible = disponible;
    }

    // Constructor para crear un libro nuevo (id se genera en BD)
    public Libro(String titulo, String isbn, Integer anio, boolean
disponible) {
        this(null, titulo, isbn, anio, disponible);
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
```

```

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    public Integer getAnio() { return anio; }
    public void setAnio(Integer anio) { this.anio = anio; }

    public boolean isDisponible() { return disponible; }
    public void setDisponible(boolean disponible) { this.disponible =
disponible; }

    @Override
    public String toString() {
        return "Libro{" +
            "id=" + id +
            ", titulo='" + titulo + "'" +
            ", isbn='" + isbn + "'" +
            ", anio=" + anio +
            ", disponible=" + disponible +
            '}';
    }
}

```

Esta clase Libro es el **modelo**: representa en Java una fila de la tabla libro. Contiene los mismos datos que guardamos en la base de datos: id, titulo, isbn, anio y disponible.

El punto clave para el CRUD es el **id**: **antes de insertar**, el libro aún no tiene id (lo genera PostgreSQL), y **después de insertar** ya tendremos un id asignado. Por eso suele existir un constructor “sin id” (para crear libros nuevos) y otro “con id” (para cuando el libro ya existe o lo hemos leído de la BD). Java elige automáticamente cuál usar según los argumentos que le pases en `new Libro(...)`.

Al estar declarados los atributos como `private` necesitamos los **getters** (`getTitulo()`, `getIsbn()`, etc.) para poder **leer datos del objeto** desde el DAO y pasarlos a los `?` del `PreparedStatement`. Importante: los getters **no leen de la base de datos**, solo **devuelven lo que ya está guardado en el objeto** en memoria.

El método `toString()` (con `@Override`) define cómo se imprime un libro por consola para que las pruebas del CRUD sean fáciles de entender.

5. CRUD con JDBC en LibroDao

Vamos a dejar un LibroDao con todos los métodos.

Ruta: src/main/java/com/dam/ad/dao/LibroDao.java

```
package com.dam.ad.dao;

import com.dam.ad.db.Db;
import com.dam.ad.model.Libro;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class LibroDao {

    private Libro mapRow(ResultSet rs) throws SQLException {
        int id = rs.getInt("id");
        String titulo = rs.getString("titulo");
        String isbn = rs.getString("isbn");
        Integer anio = (Integer) rs.getObject("anio"); // admite NULL
        boolean disponible = rs.getBoolean("disponible");
        return new Libro(id, titulo, isbn, anio, disponible);
    }

    public List<Libro> findAll() {
        String sql = "SELECT id, titulo, isbn, anio, disponible FROM libro ORDER BY id";

        try (Connection con = Db.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {

            List<Libro> libros = new ArrayList<>();
            while (rs.next()) {
                libros.add(mapRow(rs));
            }
        }
    }
}
```

```

        return libros;

    } catch (SQLException e) {
        throw new RuntimeException("Error leyendo libros", e);
    }
}

public Optional<Libro> findById(int id) {
    String sql = "SELECT id, titulo, isbn, anio, disponible FROM
libro WHERE id = ?";

    try (Connection con = Db.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setInt(1, id);

        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return Optional.of(mapRow(rs));
            }
            return Optional.empty();
        }

    } catch (SQLException e) {
        throw new RuntimeException("Error buscando libro por id=" +
id, e);
    }
}

public int insert(Libro libro) {
    String sql = "INSERT INTO libro (titulo, isbn, anio, disponible)
VALUES (?, ?, ?, ?)";

    try (Connection con = Db.getConnection();
        PreparedStatement ps = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

        ps.setString(1, libro.getTitulo());

```

```

        ps.setString(2, libro.getIsbn());
        ps.setObject(3, libro.getAnio()); // admite NULL
        ps.setBoolean(4, libro.isDisponible());

        int rows = ps.executeUpdate();
        if (rows != 1) {
            throw new SQLException("INSERT inesperado. Filas
afectadas: " + rows);
        }

        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) {
                int id = keys.getInt(1);
                libro.setId(id);
                return id;
            }
            throw new SQLException("No se devolvió id generado
(getGeneratedKeys vacío)");
        }

    } catch (SQLException e) {
        throw new RuntimeException("Error insertando libro", e);
    }
}

public boolean update(Libro libro) {
    if (libro.getId() == null) {
        throw new IllegalArgumentException("No se puede UPDATE sin
id. ¿Hiciste insert primero?");
    }

    String sql = "UPDATE libro SET titulo=?, isbn=?, anio=?,
disponible=? WHERE id=?";

    try (Connection con = Db.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, libro.getTitulo());
        ps.setString(2, libro.getIsbn());

```

```

        ps.setObject(3, libro.getAnio());
        ps.setBoolean(4, libro.isDisponible());
        ps.setInt(5, libro.getId());

        return ps.executeUpdate() == 1;

    } catch (SQLException e) {
        throw new RuntimeException("Error actualizando libro id=" +
libro.getId(), e);
    }
}

public boolean deleteById(int id) {
    String sql = "DELETE FROM libro WHERE id=?";

    try (Connection con = Db.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setInt(1, id);
        return ps.executeUpdate() == 1;

    } catch (SQLException e) {
        throw new RuntimeException("Error borrando libro id=" + id,
e);
    }
}
}

```

LibroDao es la clase que **habla con la base de datos**. Su responsabilidad es traducir operaciones de Java (crear/buscar/actualizar/borrar) a SQL.

En esta ocasión vamos a crear un método “helper” al que vamos a llamar mapRow(...). Lo creamos para **no repetir**, no tener que estar leyendo columnas del ResultSet y construir new Libro(...) todo el rato. En su lugar llamamos a mapRow(ResultSet). Este método convierte **la fila del ResultSet** en un objeto Libro (solo **la fila en la que está el cursor en ese momento**). Para convertir **todas**, necesitas while (rs.next()) y llamar a mapRow(rs) en cada vuelta.

Qué hace cada método principal:

- `findAll()` como vimos en la práctica anterior, ejecuta un `SELECT` sin parámetros y devuelve una lista de libros. Usa el método `executeQuery()` que **ejecuta una consulta SQL que devuelve filas** con los atributos consultados; te devuelve un **ResultSet**, que es la “tabla” de resultados que puedes recorrer fila a fila con `rs.next()` para guardarla en `List<Libro>`.
- `findById(id)` ejecuta un `SELECT ... WHERE id=?`. Con `(ps.setInt(1, id))` rellena el primer ? del SQL con el valor de la variable `id` que introduzcas desde el Main. Si encuentra ese id devuelve el libro (y desde Optional llamamos a `mapRow` para meterlo en la lista de libros), y si no encuentra, devuelve vacío. Sirve para comprobar rápidamente si el insert o el update han funcionado.
- `insert(libro)` ejecuta un `INSERT`. Aquí lo importante es que el id lo genera la base de datos. Por eso se prepara el statement con `RETURN_GENERATED_KEYS` y después del `executeUpdate()` que devuelve cuántas filas han sido afectadas (se espera 1 porque hemos insertado solo 1 libro), se leen las claves generadas con `getGeneratedKeys()`. El método devuelve ese id y con `libro.setId(id)` actualizamos el objeto libro en la memoria, ya con su id correspondiente.
- En `update` queremos cambiar el estado del nuevo libro a No disponible, por lo que, desde el main cambiamos el estado del libro nuevo (que creamos en el Insert anterior) a no disponible con `nuevo.setDisponible(false)` y ejecutamos `dao.update(nuevo)`. Cuando se llama a `update` este lee los datos del libro nuevo en memoria y lo actualiza en la BD, ya con el estado disponible en false. Luego, devuelve true si se actualizó 1 fila y false si no se actualizó ninguna por algún error.
- `deleteById(id)` ejecuta un `DELETE ... WHERE id=?` Recibiendo el id desde el main. Igual que `update`, devuelve true si borró una fila y false si no existía.
- `INSERT/UPDATE/DELETE` usan `executeUpdate()` que devuelve número de filas afectadas (normalmente 1 o 0).

OJO: si te falla `getGeneratedKeys`, revisa que el id sea `SERIAL/IDENTITY` y que creas el `PreparedStatement` con `Statement.RETURN_GENERATED_KEYS`.

6. Main de prueba: flujo CRUD completo

Crea una clase nueva para probar el CRUD, sin romper el Main anterior (el de findAll).

Ruta: src/main/java/com/dam/ad/app/CrudDemo.java

```
package com.dam.ad.app;

import com.dam.ad.dao.LibroDao;
import com.dam.ad.model.Libro;

public class CrudDemo {
    public static void main(String[] args) {
        var dao = new LibroDao();

        System.out.println("=== ESTADO INICIAL ===");
        dao.findAll().forEach(System.out::println);

        System.out.println("
=== INSERT ===");
        var nuevo = new Libro("Clean Code", "9780132350884-2", 2008,
true);
        int id = dao.insert(nuevo);
        System.out.println("Insertado con id=" + id + " -> " + nuevo);

        System.out.println("
=== FIND BY ID ===");
        dao.findById(id).ifPresentOrElse(
            System.out::println,
            () -> System.out.println("No encontrado")
        );

        System.out.println("
=== UPDATE (marcar no disponible) ===");
        nuevo.setDisponible(false);
        boolean updated = dao.update(nuevo);
        System.out.println("updated=" + updated);
        System.out.println("Después del update: " +
dao.findById(id).orElse(null));
    }
}
```

```

        System.out.println("
=== DELETE ===");
        boolean deleted = dao.deleteById(id);
        System.out.println("deleted=" + deleted);

        System.out.println("
=== ESTADO FINAL ===");
        dao.findAll().forEach(System.out::println);
    }
}

```

CrudDemo es un main de pruebas para ver el CRUD funcionando de forma completa y entenderlo por consola. La idea es ejecutar siempre el mismo ciclo:

1. **Listar estado inicial** con findAll() para ver lo que hay en la tabla antes de tocar nada.
2. **Insertar** un libro nuevo y guardar el id que devuelve el DAO (esto prueba que el id generado funciona).
3. **Buscar por id** con findById(id) para comprobar que el libro insertado realmente está en la base de datos.
4. **Actualizar** ese mismo libro (a no disponible) y comprobar el resultado con otro findById.
5. **Borrar** el libro por id y comprobar que ya no existe.
6. **Listar estado final** con findAll() para verificar visualmente que el CRUD dejó la tabla como esperábamos.

7. Problemas típicos y cómo resolverlos

Cuando algo falla, intenta primero reproducir el problema en pgAdmin (SQL) y luego revisa parámetros en Java.

Mensaje	Causa probable + solución
duplicate key value violates unique constraint "libro_isbn_key"	Estás insertando un isbn repetido. Cambia el isbn o borra el registro anterior.
getGeneratedKeys vacío	Falta Statement.RETURN_GENERATED_KEYS o el id no es autogenerado (revisa SERIAL/IDENTITY).
Parameter index out of range (X > number of parameters)	Tu SQL tiene menos ? de los que estás seteando, o has cambiado el orden y no coincide.
NullPointerException al setInt para anio	Si anio puede ser null, usa ps.setObject(..., libro.getAnio()) en vez de setInt.
UPDATE/DELETE devuelve 0 filas	El id no existe en la tabla (o ya lo borraste). Comprueba con SELECT WHERE id=...
org.postgresql.util.PSQLException: Connection refused / password authentication failed	Mismo diagnóstico que en sesión 1: PostgreSQL encendido, puerto 5432, usuario/clave correctos en db.properties.

8. Git (commits por hitos)

Recomendación: commits pequeños, cada vez que algo compile y funcione.

- Commit 1: "Model Libro preparado para INSERT (id nullable + getters/setters)".
- Commit 2: "LibroDao CRUD completo (findById/insert/update/delete)".
- Commit 3: "CrudDemo para probar flujo completo".

OJO: sigue sin subirse db.properties (con contraseña). Debes tener en .gitignore la línea:
src/main/resources/db.properties